

HASHING

Open Hashing · Closed Hashing · Probing Yontemleri · Beklenen Maliyet Analizi (Tam Turetme)

Temel kavramlar: m = tablo boyutu, N = eleman sayisi, $\alpha = N/m$ = yuk faktoru
 $h(x) = x \bmod m$ = hash fonksiyonu

1. Open Hashing (Separate Chaining)

Her slot bir bagli liste. Cakisan elemanlar zincire eklenir. $\alpha > 1$ olabilir.

Open Hashing – Separate Chaining

put / get

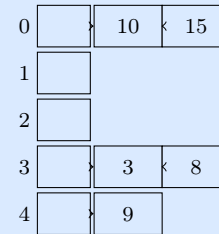
```
public class OpenHashing<K,V> {
    private static class Node<K,V>{
        K key; V value; Node<K,V> next;
        Node(K k,V v){key=k;value=v;}
    }
    private final int m;
    private final Node<K,V>[] table;

    public OpenHashing(int m){
        this.m=m; table=new Node[m];
    }
    private int hash(K key){
        return (key.hashCode()
            & 0x7fffffff) % m;
    }
    public void put(K key,V value){
        int i=hash(key);
        for(Node<K,V> n=table[i];
            n!=null; n=n.next){
            if(n.key.equals(key)){
                n.value=value; return;}
        }
        Node<K,V> nd=
            new Node<>(key,value);
        nd.next=table[i];
        table[i]=nd; // basa ekle
    }
    public V get(K key){
        for(Node<K,V> n=
            table[hash(key)];
            n!=null; n=n.next){
            if(n.key.equals(key))
                return n.value;
        }
        return null;
    }
}
```

delete

```
public void delete(K key){
    int i=hash(key);
    if(table[i]==null) return;
    if(table[i].key.equals(key)){
        table[i]=table[i].next;
        return;
    }
    for(Node<K,V> n=table[i];
        n.next!=null; n=n.next){
        if(n.next.key.equals(key)){
            n.next=n.next.next;
            return;
        }
    }
}
```

Gorsel – $m = 5$



Ortalama: $O(1 + \alpha)$ En kotü: $O(n)$

2. Closed Hashing (Open Addressing)

Tüm elemanlar tablonun icinde. Cakisma olunca baska slot aranir. $\alpha < 1$ zorunlu.

Closed Hashing – Open Addressing

Linear: $p(k, i) = (h(k) + i) \bmod m$

Quadratic: $p(k, i) = \left(h(k) + \frac{i^2+i}{2}\right) \bmod m$

Double Hash: $p(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

Linear Probing – put / get

```
public class LinearProbing<K,V>{
    static class Entry<K,V>{
        K key; V value;
        boolean deleted;
        Entry(K k,V v){key=k;value=v;}
    }
    private final int m;
    private Entry<K,V>[] table;
    private int N;

    private int hash(K key){
        return (key.hashCode()
            & 0x7fffffff)%m;
    }
    public boolean put(K key,V val){
        if(N>=m) return false;
        int i=hash(key);
        while(table[i]!=null
            && !table[i].deleted){
            if(table[i].key.equals(key)){
                table[i].value=val;
                return true;
            }
            i=(i+1)%m;
        }
        table[i]=new Entry<>(key,val);
        N++; return true;
    }
    public V get(K key){
        int i=hash(key);
        while(table[i]!=null){
            if(!table[i].deleted &&
                table[i].key.equals(key))
                return table[i].value;
            i=(i+1)%m;
        }
        return null;
    }
}
```

delete – Lazy Deletion

```
public boolean delete(K key){
    int i=hash(key);
    while(table[i]!=null){
        if(!table[i].deleted &&
            table[i].key.equals(key)){
            table[i].deleted=true;
            N--; return true;
        }
        i=(i+1)%m;
    }
    return false;
}
```

Neden Lazy Deletion?

Gerçekten silseydin: [A] [] [] [B] iken
A'yi silersen boşluk oluşur ve B'yi arayan
get null döner – oysa B hala tabloda.
deleted=true koyu arama zinciri sağlar.

Gorsel – $m = 7$

	[0]
	[1]
	[2] 20=6→4
10	[3] 17=3→5
20	[4]
17	[5]
27	[6]

Pratik – Probing

(a) $m = 11$, Linear ile {10,22,31,4,15,28,17,88,59} – her slotu yazın.

(b) Lazy deletion olmadan delete(A) sonrası get(B) neden fail eder? Somut örnek verin. _____

3. Analiz: Beklenen Maliyet – Adim Adim

Analiz: Beklenen Karsilastirma Sayisi – Adim Adim Turetme

Adim 1 – Binomial dagilim: tam k eleman gorme olasiligi

N elemanın m slota uniform dagildigi varsayalım. Bir slotta tam k eleman olma olasiligi binomial:

$$P_k = \binom{N}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{N-k}$$

Ne anlama geliyor? N elemandan k tanesini sececiz ($\binom{N}{k}$), her biri bu slota $(1/m)$ olasilikla, gerisi baska slota $(1 - 1/m)$ olasilikla dustugu durum.

Adim 2 – Poisson’a yakinsama ($N \rightarrow \infty$, $\alpha = N/m$ sabit)

$$\binom{N}{k} \left(\frac{\alpha}{N}\right)^k \left(1 - \frac{\alpha}{N}\right)^{N-k} \xrightarrow{N \rightarrow \infty} \frac{e^{-\alpha} \alpha^k}{k!}$$

Neden? N cok buyuk, m cok buyuk, ama $\alpha = N/m$ sabit kalsin. O zaman $(1 - \alpha/N)^N \rightarrow e^{-\alpha}$ ve $N(N-1) \cdots (N-k+1)/N^k \rightarrow 1$. Yani cok buyuk tablolarla slot dolulugu Poisson dagilimine uyar. Pratikte $\alpha \leq 1$ iken bu yaklasim cok iyidir.

Adim 3 – Basarisiz arama / ekleme maliyeti (Open Hashing)

Eleman bulunamazsa tum zinciri taramak gerekir. Zincir uzunlugu k ise k karsilastirma yapilir.

$$E[\text{karsilastirma}] = \sum_{k=0}^{\infty} P_k \cdot k = \sum_{k=0}^{\infty} \frac{e^{-\alpha} \alpha^k}{k!} \cdot k$$

$k = 0$ terimi sifir, $k \geq 1$ icin $k/k! = 1/(k-1)!$:

$$= e^{-\alpha} \sum_{k=1}^{\infty} \frac{\alpha^k}{(k-1)!} = e^{-\alpha} \cdot \alpha \sum_{j=0}^{\infty} \frac{\alpha^j}{j!} \underset{\text{Taylor: } e^{\alpha}}{=} e^{-\alpha} \cdot \alpha \cdot e^{\alpha} = \boxed{\alpha}$$

Sezgi: Zincirin beklenen uzunlugu α . Bulamayınca tum zinciri geziyoruz. Ekleme de ayni: nereye koyacagimizi bulmak icin zinciri gezeriz.

Adim 4 – Basarili arama maliyeti (Open Hashing)

Aranan eleman k uzunluklu bir zincirin i . elemaniysa i adim gerekir. Zincirde her pozisyon esit olasilikla, yani beklenen pozisyon $(k + 1)/2$:

$$E[\text{pozisyon}] = \frac{k + 1}{2}$$

Bunu tum zincir uzunluklari uzerinden ortalarsak:

$$E[\text{basarili}] = \sum_{k=1}^{\infty} P_k \cdot \frac{k + 1}{2} = \frac{1}{2} \left(\sum_{k=1}^{\infty} k P_k + \sum_{k=1}^{\infty} P_k \right) \approx \frac{1}{2}(\alpha + 1) = \boxed{1 + \frac{\alpha}{2}}$$

Sezgi: Bulunca zincirin yarisini gezmis oluyoruz. $\alpha = 0.7$ icin beklenen adim sadece 1.35 – neredeyse sabit!

Adim 5 – Closed hashing: bir sonraki slot bos mu? Olasilik turetme

N eleman var, m slot var. i probe sonrasi hepsini dolu gorduk. $(i + 1)$. probe'da bos slot gorme olasiligi:

$$\frac{m - N}{m - i} = \frac{m - N}{m - i} \approx \frac{m - N}{m} = 1 - \alpha \quad (N \gg i \text{ oldugunda})$$

Ne anlama geliyor? i dolu slot gordukten sonra geri kalan $m - i$ slotun $m - N$ tanesi bos. Yani i . probe'dan sonra bos slot olasiligi yaklasik $1 - \alpha$. Bu demek oluyor ki her probe birbirinden bagimsiz gibibasitten α olasilikla dolu.

Adim 6 – Basarisiz arama / ekleme (Closed Hashing – Uniform Probing)

Her probe'da bos slot bulma olasiligi $1 - \alpha$, dolu olma α . Tam i dolu gordukten sonra $i + 1$. probe'da bos bulma olasiligi: $\alpha^i(1 - \alpha)$. Toplam beklenen probe sayisi (bos slot bulana kadar kac probe):

$$E[\text{probe}] = \sum_{i=0}^{\infty} (i + 1) \cdot \alpha^i \cdot (1 - \alpha) = (1 - \alpha) \sum_{i=0}^{\infty} (i + 1) \alpha^i = (1 - \alpha) \cdot \frac{1}{(1 - \alpha)^2} = \boxed{\frac{1}{1 - \alpha}}$$

Geometrik serinin turevinden: $\sum_{i=0}^{\infty} (i + 1) x^i = \frac{1}{(1 - x)^2}$.

Sezgi: $\alpha = 0.5$ ise ortalama 2 probe, $\alpha = 0.9$ ise 10 probe. Tablo dolunca patlar!

Adim 7 – Basarili arama / silme (Closed Hashing – integral ile)

N eleman eklenmistir. i . eleman eklenirken tablo dolulugu i/m idi. O anda ekleme maliyeti $\frac{1}{1-i/m}$ idi (Adim 6). Basarili aramanin maliyeti = o elemani *ekledigimiz anki* maliyetin ortalamasi:

$$E[\text{basarili}] = \frac{1}{N} \sum_{i=0}^{N-1} \frac{1}{1-i/m} = \frac{1}{N} \sum_{i=0}^{N-1} \frac{m}{m-i}$$

N cok buyukse bu toplam integralle yaklastirilir ($x = i/m$, $dx = 1/m$):

$$\approx \frac{1}{\alpha} \int_0^{\alpha} \frac{1}{1-x} dx = \frac{1}{\alpha} \left[-\ln(1-x) \right]_0^{\alpha} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha} = \left[\frac{1}{\alpha} \ln \frac{1}{1-\alpha} \right]$$

Neden integral? Cok fazla terim var, toplam integralle guzce kapaniyor. $\alpha = 0.5$ icin: $\frac{1}{0.5} \ln 2 \approx 1.39$ adim. Guzel!

Adim 8 – Linear Probing icin Knuth Formulleri (Kumelenme Etkisi)

Linear probing’de ardisik dolu slotlar kume (cluster) olusturur. Bu yuzden uniform probing’den daha kotu performans verir. Knuth “Art of Computer Programming” Vol.3’te ispatladi:

$$\begin{aligned} \text{Ekleme / Basarisiz:} & \quad \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right) \\ \text{Basarili / Silme:} & \quad \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right) \end{aligned}$$

Neden daha kotu? Dolu slot gorundugunde bir sonraki slot da cogunlukla dolu cunku aynı kumenin parçasi. Yani bagimli – basit geometrik seri yapmaz. Bu formullerin ispatı cok daha uzun (Markov zinciri ile). Ama sonuc mantikli: $\alpha \rightarrow 1$ iken her iki formül de sonsuza gider.

Yontem	α	Basarili	Basarisiz/Ekleme	Adim
Open Chaining	≥ 0	$1 + \alpha/2$	α	3,4
Uniform Probing	< 1	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$	$\frac{1}{1-\alpha}$	6,7
Linear Probing	< 1	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$	8

α	Chain. basarili	Unif. basarili	Lin. basarili	Lin. ekleme
0.50	1.25	1.39	1.50	2.50
0.75	1.38	1.85	2.50	8.50
0.90	1.45	2.56	5.50	50.5

Pratik – Hesaplama

(a) $\alpha = 0.6$ için Open Chaining basarili/basarisiz, Uniform Probing basarili/basarisiz deger hesaplayin (4 sayi). _____

(b) Adim 3'teki turetime gore: $\sum_{k=1}^{\infty} \frac{k}{k!} \alpha^k = \alpha e^{\alpha}$ oldugunu $k/k! = 1/(k-1)!$ kullanarak gosterin. _____

(c) Adim 6'da geometrik serinin turevini kullandik: $\sum_{i=0}^{\infty} (i+1)x^i = \frac{1}{(1-x)^2}$. Bu esitligi $\sum x^i = \frac{1}{1-x}$ 'in her iki tarafini x 'e gore turevleyerek elde edin. _____

(d) Neden linear probing'de $\alpha = 0.9$ iken ekleme maliyeti 50'ye firliyor? _____

Secim Rehberi:

- $\alpha > 1$ gerekiyorsa veya dinamik yuk varsa: **Open Chaining**
- Cache performansi onemli, $\alpha < 0.7$: **Linear Probing**
- Kumelenmeyi azaltmak: **Quadratic** (m asal, $\alpha < 0.5$) veya **Double Hashing**
- **Altin kural:** $\alpha > 0.7$ olunca rehash yap (boyutu ikiyle carp, yeniden dagit)